

PDDL Modeling for WorkBenchMark: Classical Planning for LEGO Duplo Assembly in TAMPanda

Moritz Wehr, Antonina Putowska

AI for Robotics (Summer Semester 2026)

Chair of Machine Learning and Reasoning, RWTH Aachen University

Repository: github.com/IX0Y3/PDDL-Modeling-for-WorkBenchMark

Abstract

WorkBenchMark provides LEGO Duplo assembly tasks and an Assembly-by-Disassembly (ABD) baseline. In this project we design a PDDL domain for vertical stacking, compile WorkBenchMark YAML task specs into a PDDL problem and then plan with a general purpose pddl planner. Afterwards we execute the resulting action sequences in TAMPanda via `DomainBridge`. This report describes the modeling choices, the design of the YAML to PDDL to plan to simulation pipeline. At the end we evaluate the project by solving Tier 1–2 tasks and comparing planning success, execution success, planning time, and plan length against the given ABD baseline.

1 Introduction

Robotic assembly requires deciding task sequencing and motion planning/execution. WorkBenchMark [1] focuses on LEGO Duplo: Each task specifies an initial brick layout and a target assembly, and a planner must produce a feasible assembly order. The benchmark ships an ABD baseline that derives assembly sequences from geometrically valid disassembly.

Project P1 takes a different angle. Instead of implementing ABD, we model the assembly problem in PDDL and use an already existing planner. The difficulty here is modelling. Which facts belong in the symbolic state? Which geometric constraints become preconditions and which become external feasibility checks? After creating the PDDL Domain we parse the YAML Configurations and generate a plan using the already mentioned planner. Generated plans are executed in the courses MuJoCo environment TAMPanda through `DomainBridge`.

Goals

- (1) Design a PDDL domain for Duplo pick/place/stack on Tier 1–2 tasks.
- (2) Build a compiler/parser from WorkBenchMark YAML to PDDL problems.
- (3) Integrate a classical planner
- (4) Evaluate classical planning against ABD on planning time and plan quality, with a failure analysis of the symbolic model.

2 Background

WorkBenchMark contains hundreds of LEGO Duplo assembly tasks across four complexity tiers. Each Task has a YAML specification listing bricks, poses, and assembly goals [1]. ABD is the reference planner. Our goal is not to beat ABD, but to check how good a PDDL model goes on the simplest assemblies.

TAMPanda provides simulation, motion, and an execution bridge. We reuse the blocks world pattern. PDDL actions map to pick/place motion calls. TAMPanda itself is treated as a dependency. We do not modify or optimize it.

3 Scope

Following the project sheet, we restrict the projects scope to

- Tier 1 (2-brick) and Tier 2 (3–5 brick) vertical stacks
- Tier 3–4 structures and perception as **out** of scope.
- ground-truth poses from YAML (no GroundingDINO / SAM / FoundationPose)
- MuJoCo simulation only (no real robot)

4 Approach

4.1 Pipeline

Below is the planned order of the execution pipeline of our project:

```
YAML task → PDDL problem → planner → action plan → TAMPanda execution.
```

Planning uses the already existing planer `pyperplan`.

4.2 PDDL Domain Design

The core design question is symbolic granularity. Stud-level grid positions capture LEGO geometry more precise but explode the state space fast. Also they require precise stud alignment facts. We therefore want to design an *area-based* model. This type of model only tracks regions (pick area, assembly area, ...) and stacking relations, not exact stud coordinates. The area based PDDL Domain will have the following types, predicates and actions.

Types.

- `brick` — a Duplo brick to pick, place, or stack
- `gripper` — the robots end-effector
- `area` — a workspace region (pick area, assembly area)

Predicates.

- `stacked-on` — one brick is stacked on another brick
- `on-table` — one brick is on the table
- `clear` — nothing is stacked on this brick
- `graspable` — one brick may be picked
- `in-pick-area` — one brick is in the pick region
- `in-assembly-area` — one brick is in the assembly region
- `target-pose` — one brick belongs to the goal assembly
- `holding` — gripper currently holds a brick
- `gripper-empty` — gripper holds nothing

Actions.

- `pick` — grasp a clear brick from the pick area
- `place` — put a held brick onto the assembly base
- `stack` — put a held brick onto another brick in the assembly area

4.3 Level of abstraction

This PDDL domain does not model exact positions, stud alignment, or how the arm approaches a brick. That is handled later by TAMPanda’s integrated motion planner. Symbolically we only care about a sensible stacking order. The top of the bricks must be clear, the base must be in the assembly area, and the gripper can hold one brick at a time. We keep the model simple so planning stays easy on Tier 1–2, and then check in simulation if this abstraction is enough and how it performs.

4.4 YAML to PDDL parser

Each WorkBenchMark task YAML lists bricks with initial poses and a target stack structure. The PDDL problem compiler/parser will:

1. instantiate one PDDL object per brick plus a gripper
2. assert initial predicates (`in-pick-area`, `clear`, ...) from the initial layout
3. encode the goal as the required `stacked-on` chain and what brick base has to be on the table (implied by the target assembly).

4.5 Execution Bridge

After planning, each grounded action is mapped to a `PickPlaceExecutor` call through `DomainBridge`, following TAMPanda’s blocks-world bridge. Predicate grounding uses YAML ground-truth poses. Execution success requires every action to complete without motion or grasp failure.

5 Evaluation Plan

We evaluate on Tier 1 and Tier 2 vertical stacks with $N=20$ tasks per tier (project specification). We observe the following metrics:

- **Planning success:** fraction of tasks with a valid plan.
- **Execution success:** fraction where all plan steps succeed in TAMPanda.
- **Planning time and plan length:** compared to ABD where available.
- **Failure analysis:** which structures expose model gaps (e.g. incorrect clear/stack constraints).

References

- [1] W. Ma, D. Swoboda, M. Tschesche, and T. Hofmann, “WorkBenchMark: A LEGO-Based Assembly Benchmark with an Assembly-by-Disassembly Baseline for the Smart Manufacturing League,” *arXiv preprint arXiv:2606.19358*, 2026. <https://arxiv.org/pdf/2606.19358>, <https://workbenchmark.github.io>.

Repository. <https://github.com/IX0Y3/PDDL-Modeling-for-WorkBenchMark>.